

Deepseek Chat



regex_crawler_1.py

PY 3.37KB

Please modify the attached Python script so it saves the entire article in an output entry provided, at least one REGEX matches in that article! Otherwise, preserve all functionality!



Thought for 141seconds ^

We are going to modify the script to include the entire article text in the XML entry only if there is at least one regex match in the article.

However, note that the current structure of the XML is:

```
<matches>
  <entry file="..." path="...">
    <match pattern="...">sentence</match>
    ...
  </entry>
  ...
</matches>
```

We are going to add an attribute to the <entry> element for the entire article text, but note that we don't want to change the existing structure too much.

We can add an attribute called 'article' to the entry element that contains the entire article text? But note that the article text might be large and contain special characters.

Alternatively, we could add a new child element for the entire article? Let me see what the requirements says: "saves the entire article in an output entry"

Since the requirements says "output entry", we can interpret that as the <entry> element. We can add a new attribute or a new child.

However, note that the requirements says: "at least one REGEX matches in that article". So we are only going to include the entire article text if there is at least one match.

But note: currently, we are only writing an entry if there is at least one match (because `write\_xml\_entry` does nothing if there are no matches). So we are already only processing files with matches.

Therefore, we can change the `write_xml_entry` function to include the entire article text.

How about we add a new child element for the entire article? For example:

```
<entry file="..." path="...">
  <article> ... entire article text ... </article>
  <match pattern="...">sentence</match>
  ...
</entry>
```

But note: the article text is the cleaned text (without script and style) that we have already extracted? Yes, we have the `text` variable in `process_html_file`.

However, we are currently not returning the entire article text from `process_html_file`. We are returning only the matches (each match is a sentence and the pattern name).

We need to change `process_html_file` to also return the entire article text? But note: we are only going to use the entire article text in the XML if there are matches.

Alternatively, we can do:

In `process_directory`:

```
matches = process_html_file(...) # currently returns a list of matches
```

We can change `process_html_file` to return `(matches, article_text)` and then in `process_directory` when we call `write_xml_entry` we pass the `article_text`.

But note: the requirement is to preserve all functionality. We are only adding the entire article when there's at least one match.

Let's break down the changes:

1. Modify `process_html_file` to return both the matches and the entire article text (the cleaned text without script and style).

Currently, we are returning `matches` (a list). We'll change that to return `(matches, text)`.

2. In the `process_directory` function, when we call `process_html_file`, we now get two values: `matches, article_text = process_html_file(...)`

3. Then, when we call `write_xml_entry`, we also pass the `article_text`.

4. In `write_xml_entry`, we will create a child element for the entire article. But note: we are writing the XML as we go. We are writing each entry as we process each file.

We change the function to: `write_xml_entry(xml_file, file_path, matches, article_text)`

Then, in that function, after creating the `entry` element, we add a child element for the article:

```
article_elem = ET.SubElement(entry, 'article')
```

```
article_elem.text = article_text # but note: the article text may contain special
```

characters and newlines. We are using `ET.tostring` which should handle that?

However, note that the article text is a long string and may contain characters that need to be escaped. The `ElementTree` should take care of that.

5. But note: what if the article text is very long? We are writing the entire text as a single string. That should be acceptable.

6. Also, we must note that we are currently writing the entry element as a string and then writing that string to the file. We are not building a full tree in memory.

We are building each entry as an `ElementTree` element and then converting to string. This is acceptable.

7. However, the requirements says: "at least one REGEX matches in that article". We are already not writing an entry if there are no matches. So we are safe.

8. But what about the existing matches? We are still writing the `<match>` elements as before.

9. Also, we must update the main function to handle the new return value from ``process_html_file``.

However, note that in the ``process_html_file`` function, if there's an exception, we return `[]`. Now we have to return two values? We can return `([], None)` or `([], '')`? But then in ``process_directory`` when we call ``write_xml_entry`` we would pass that empty string? And then in ``write_xml_entry`` we would add an empty article element? But note: if there's an error, we return `[]` and then ``write_xml_entry`` does nothing (because if not matches: return). So we are safe.

We can change the error return to `([], None)` and then in ``process_directory``:

```
matches, article_text = process_html_file(...)
if matches:
    write_xml_entry(f, full_path, matches, article_text)
```

But note: what if we have an exception? We return `[]` and then we skip writing the entry. So we don't need to pass the `article_text` in that case.

10. So let's change the ``process_html_file`` function:

```
try:
    ...
    return matches, text # instead of return matches
except Exception as e:
    print(...)
    return [], None # or return [], '' but then we have to handle it in the caller
```

11 In ``process_directory``:

for ...:

```
if filename.lower().endswith('.html'):
    full_path = ...
    matches, article_text = process_html_file(full_path, regexes)
    if matches:
        write_xml_entry(f, full_path, matches, article_text)
```

12. Then change `write_xml_entry` to take an additional argument `article_text` and add a child element for the article.

But note: what if the `article_text` is `None`? That would happen in case of error. But we are only calling `write_xml_entry` when `matches` is non-empty. However, we are returning `([], None)` on error, so we won't call `write_xml_entry` because `matches` is empty. So we are safe.

However, in the normal case, we have `article_text` as a string. But what if the `article_text` is an empty string? Then we just write an empty `<article></article>`? That's acceptable.

13. Also, note that the article text we are using is the one we extracted (without script and style) and then we split it into sentences? We are using the same text for the entire article.

14. However, note that the text extraction by BeautifulSoup might have introduced some changes (like replacing tags with spaces). We are using `get_text(separator=' ', strip=True)`. That text is what we want.

15. Let's update the functions accordingly.

But note: the requirements says "preserve all functionality". We are adding the entire article only when there is at least one match, and we are keeping the existing matches.

However, note that the current XML structure is:

```
<entry ...>
  <match ...> ... </match>
  ...
</entry>
```

We are going to change it to:

```
<entry ...>
  <article> ... full text ... </article>
  <match ...> ... </match>
  ...
</entry>
```

This change in the XML structure might break existing consumers? But the requirement does not specify that the XML structure must remain unchanged. It says "preserve all functionality", meaning the existing matching functionality. And we are adding the entire

article.

However, if the requirement is to not break existing consumers, we might reconsider. But the requirement says "saves the entire article in an output entry", so we have to save it.

Since the problem does not specify the exact XML structure, we'll go with adding a new child element.

Alternatively, we could add the entire article as an attribute? But that would be very long and not appropriate.

We'll go with a child element.

Implementation

In `write_xml_entry`:

```
entry = ET.Element('entry', {'file': os.path.basename(file_path), 'path': file_path})

# Add article as a child element
article_elem = ET.SubElement(entry, 'article')
article_elem.text = article_text # This is the entire article string

# Then add the matches
for match in matches:
    ...
```

16. However, note that the article text might contain newlines and extra spaces. We are not altering it. We are writing it as is.

17. Also note that the existing matches are extracted from the same article text? Actually, we split the article text into sentences. But the article element will have the entire text without splitting.

18. We must be cautious about memory: if the article text is very large, we are holding it in memory. But that was already the case.

19. Let's update the code accordingly.

Steps:

a) Change the return type of `process_html_file` to `(matches, article_text)` and adjust the return in the try block and the except block.

b) In `process_directory`, when we get the result from `process_html_file`, we check if there are matches. If there are, we call `write_xml_entry` with the `article_text`.

c) Change `write_xml_entry` to take an extra argument `article_text` and create the `<article>` element.

d) Also, note that the `article_text` might be `None` in the error case? But we are only writing when there are matches, and in the error case we return `[]` for matches, so we skip. So in `write_xml_entry` we assume `article_text` is a string (or at least something that can be

converted to a string). But what if we have an error and we still call `write_xml_entry`? We don't because we only call when there are matches. So we are safe.

20. But what about the error case in `process_html_file`? We return `([], None)`. Then in `process_directory`, we skip the write. That's consistent with the old behavior (we didn't write an entry for that file).

21. Let's code accordingly.

However, note: what if the file is processed successfully but there are no matches? We return `([], article_text)`. Then we skip writing the entry. So we don't write the `article_text`. That's what we want.

22. So we are good.

Let's write the changes.

I'll modify the script to include the entire article text in the XML output when there's at least one regex match. Here are the key changes:

1. Return both matches and article text from `process_html_file`
2. Pass article text to `write_xml_entry`
3. Add `<article>` element to XML output when matches exist

Here's the modified script:

python

 Copy  Download

```
import os
import re
import argparse
from bs4 import BeautifulSoup
import xml.etree.ElementTree as ET
from xml.dom import minidom

def process_html_file(file_path, regexes):
    """Process a single HTML file and return matches for all regexes along with article text"""
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            soup = BeautifulSoup(f, 'html.parser')

            # Remove script and style elements
            for script in soup(["script", "style"]):
                script.decompose()

            # Get all text and split into sentences (very simple approach)
            text = soup.get_text(separator=' ', strip=True)
            sentences = re.split(r'(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|?|!)\s', text)

            matches = []
            for sentence in sentences:
                for name, pattern in regexes.items():
```

```

        if pattern.search(sentence):
            matches.append({
                'sentence': sentence.strip(),
                'pattern': name
            })
        return matches, text # Return both matches and full article text
except Exception as e:
    print(f"Error processing {file_path}: {str(e)}")
    return [], None

def write_xml_entry(xml_file, file_path, matches, article_text):
    """Write matches and article text for a single file to the XML file"""
    if not matches:
        return

    # Create entry element
    entry = ET.Element('entry', {'file': os.path.basename(file_path), 'path': file_path})

    # Add article text as a child element
    article_elem = ET.SubElement(entry, 'article')
    article_elem.text = article_text

    for match in matches:
        match_elem = ET.SubElement(entry, 'match', {'pattern': match['pattern']})
        match_elem.text = match['sentence']

    # Write to file
    xml_str = ET.tostring(entry, encoding='unicode')
    xml_file.write(xml_str + '\n')

def process_directory(root_dir, output_file, regexes):
    """Process all HTML files in directory and subdirectories"""
    with open(output_file, 'w', encoding='utf-8') as f:
        # Write XML header
        f.write('<?xml version="1.0" encoding="UTF-8"?>\n')
        f.write('<matches>\n')

        # Process each HTML file
        for dirpath, _, filenames in os.walk(root_dir):
            for filename in filenames:
                if filename.lower().endswith('.html'):
                    full_path = os.path.join(dirpath, filename)
                    matches, article_text = process_html_file(full_path, regexes) # Get both values

                    if matches: # Only write entries with matches
                        write_xml_entry(f, full_path, matches, article_text)

        # Close root element
        f.write('</matches>\n')

def main():
    # Define your regex patterns here
    regex_patterns = {
        'regex1': re.compile(r'\b(w+)\b\s+und\s+\b1(?:innen|nnen)\b' re.IGNORECASE)
    }

```

```

    'regex2': re.compile(r'\b\w*.*\w*\b', re.IGNORECASE),
    'regex3': re.compile(r'\b\w*:\w*\b', re.IGNORECASE),
    'regex4': re.compile(r'\b\w*_\w*\b', re.IGNORECASE)
    # Add more patterns as needed
}

# Set up argument parser
parser = argparse.ArgumentParser(
    description='Process German newspaper HTML files and extract sentences matchi
ng given patterns.')
parser.add_argument('input_dir', help='Directory containing HTML files')
parser.add_argument('output_file', help='XML output file path')

args = parser.parse_args()

print(f"Processing directory: {args.input_dir}")
print(f"Output will be written to: {args.output_file}")

process_directory(args.input_dir, args.output_file, regex_patterns)
print("Processing complete.")

if __name__ == '__main__':
    main()

```

Key modifications:

1. `process_html_file` now returns both matches and the full article text
2. `write_xml_entry` now takes an additional `article_text` parameter and adds an `<article>` element to the XML entry
3. `process_directory` checks if there are matches before writing the XML entry
4. All original functionality is preserved - only files with matches are processed and output

The XML output will now include the full article text in an `<article>` element whenever there's at least one regex match in the article.